Rockchip Linux6.12 SDK 快速入门

文档标识：RK-JC-YF-A67

发布版本：V1.0.1

日期：2026-06-29

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福州市铜盘路软件大道89号18号楼

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要描述了Linux6.12 SDK的基本使用方法，旨在帮助开发者快速了解并使用Linux6.12通用SDK开发包。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

各芯片系统支持状态

芯片名称	U-Boot版本	Kernel版本	Buildroot版本	Yocto版本	Debian版本
RK3572	v5	6.12	2026.02	6.0	13
RK3576	v5	6.12	2026.02	6.0	13
RK3588	v5	6.12	2026.02	6.0	13

修订记录

日期	版本	作者	修改说明
2026-03-20	V0.0.1	Weiye Chen	初始版本
2026-03-30	V0.0.2	Weiye Chen	1.删除硬件开发指南 2.删除芯片资料 3.修正部分文件路径
2026-04-20	V0.1.0	Weiye Chen	更新Beta版本
2026-06-20	V1.0.0	Weiye Chen	更新Release版本
2026-06-29	V1.0.1	Weiye Chen	更新 updateEngine 升级指南

目录

Rockchip Linux6.12 SDK 快速入门

1. SDK预编译镜像
2. 开发环境搭建
   - 2.1 准备开发环境
   - 2.2 安装库和工具集
     - 2.2.1 检查主机的 `python` 版本
     - 2.2.2 检查主机的 `make` 版本
     - 2.2.3 检查主机的 `lz4` 版本
3. Docker环境搭建
4. 软件开发指南
   - 4.1 开发向导
   - 4.2 Buildroot开发指南
   - 4.3 Debian开发指南
   - 4.4 第三方OS移植
   - 4.5 DPDK开发指南
   - 4.6 实时性Linux开发说明
   - 4.7 RKIPC开发指南
   - 4.8 Rockchip Linux updateEngine升级指南
     - 4.8.1 Recovery 模式 OTA 升级
     - 4.8.2 Linux A/B 模式 OTA 升级
     - 4.8.3 差分升级
     - 4.8.4 SD 卡启动盘升级
   - 4.9 Rockchip Linux ROS2 软件开发指南
     - 4.9.1 编译 ROS2 依赖包
     - 4.9.2 准备 Docker 环境
     - 4.9.3 在 Docker 中交叉编译 ROS2
     - 4.9.4 打包 rootfs 并验证
   - 4.10 Rockchip IDC 软件开发指南
     - 4.10.1 编译配置
     - 4.10.2 运行示例
   - 4.11 RKNN 开发指南
     - 4.11.1 RKNPU2 平台（RK3576、RK3588 等）
     - 4.11.2 RKNPU3 平台（RK3572）
   - 4.12 RKAI 开发指南
     - 4.12.1 编译配置
     - 4.12.2 模型文件
     - 4.12.3 LLM 运行示例
     - 4.12.4 VLM 运行示例
   - 4.13 软件更新记录
5. SDK 工程目录介绍
6. SDK交叉编译工具链介绍
   - 6.1 U-Boot 及Kernel编译工具链
   - 6.2 Buildroot工具链
     - 6.2.1 配置编译环境
     - 6.2.2 打包工具链
   - 6.3 Debian工具链
   - 6.4 Yocto工具链
7. SDK编译说明
   - 7.1 SDK编译命令查看
   - 7.2 SDK板级配置
     - 7.2.1 芯片选择
     - 7.2.2 板级配置选择

7.3 SDK定制化配置
7.4 全自动编译
7.5 各模块编译
7.5.1 U-Boot编译
7.5.2 Kernel编译
7.5.3 Recovery编译
7.5.4 Buildroot编译
7.5.4.1 Buildroot模块编译
7.5.5 Debian编译
7.5.6 Yocto 编译
7.6 固件的打包
8. 刷机说明
8.1 Windows 刷机说明
8.2 Linux 刷机说明
8.3 系统分区说明

1. SDK预编译镜像

使用Linux SDK预编译镜像，开发人员可以省去从源代码编译整个操作系统的过程，直接将预编译的镜像刷入开发板，从而快速启动开发和进行相关评估、对比，可减少因编译问题导致的开发时间和成本浪费。

可从公开地址下载 SDK固件下载点击这里

固件路径：**通用 Linux SDK 固件**-> **Linux6.12**

如果需要修改SDK代码或快速入门，请参考下面章节。

2. 开发环境搭建

2.1 准备开发环境

我们推荐使用 Ubuntu 22.04 或更高版本的系统进行编译。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。
硬件要求：64 位系统，硬盘空间大于 40G。如果您进行多个构建，将需要更大的硬盘空间。
软件要求：Ubuntu 22.04 或更高版本系统。

2.2 安装库和工具集

使用命令行进行设备开发时，可以通过以下步骤安装编译SDK需要的库和工具。

使用如下apt-get命令安装后续操作所需的库和工具：

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool zstd expect expect-dev g++ patchelf chrpath gawk texinfo \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python3-setuptools swig python3-dev
```

> **说明：**
> 安装命令适用于Ubuntu22.04，其他版本请根据安装包名称采用对应的安装命令，若编译遇到报错，可以视报错信息，安装对应的软件包。其中：
>
> - python 要求安装 python 3.6 及以上版本，Ubuntu 22.04 默认安装 python 3.10。
> - make 要求安装 make 4.0 及以上版本，Ubuntu 22.04 默认安装 make 4.3。
> - lz4 要求安装 lz4 1.7.3 及以上版本，Ubuntu 22.04 默认安装 lz4 1.9.3。
> - 编译Yocto需要VPN网络。

2.2.1 检查主机的python版本

检查主机的python版本方法如下：

- 检查主机python版本

```
$ python3 --version
Python 3.10.6
```

如果不满足 python 3.6 及以上版本要求，请根据主机系统版本安装对应的软件包。

2.2.2 检查主机的make版本

检查主机的make版本方法如下：

- 检查主机make版本

```
$ make -v
GNU Make 4.3
Built for x86_64-pc-linux-gnu
```

如果不满足 make 4.0 及以上版本要求，请根据主机系统版本安装对应的软件包。

2.2.3 检查主机的lz4版本

检查主机的lz4版本方法如下：

- 检查主机lz4版本

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

如果不满足 lz4 1.7.3 及以上版本要求，请根据主机系统版本安装对应的软件包。

3. Docker环境搭建

为帮助开发者快速完成上面复杂的开发环境准备工作，我们也提供了交叉编译器Docker镜像，以便客户可以快速验证，从而缩短编译环境的构建时间。

使用Docker环境前，可参考如下文档进行操作
<SDK>/docs/cn/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_CN.pdf。

已验证的系统如下：

发行版本	Docker 版本	镜像加载	固件编译
Ubuntu 22.04	24.0.5	pass	pass

发行版本	Docker 版本	镜像加载	固件编译
Ubuntu 21.10	20.10.12	pass	pass
Ubuntu 21.04	20.10.7	pass	pass
Ubuntu 18.04	20.10.7	pass	pass
Fedora 35	20.10.12	pass	NR (not run)

Docker镜像可从网址 Docker镜像 获取。

4. 软件开发指南

4.1 开发向导

为帮助开发工程师更快上手熟悉 SDK 的开发调试工作，随 SDK 发布《Rockchip_Developer_Guide_Linux_Software_CN.pdf》，可在`docs/cn/`目录下获取，并会不断完善更新。

4.2 Buildroot开发指南

为帮助开发工程师更快上手熟悉Buildroot系统的开发调试，随 SDK 发布《Rockchip_Developer_Guide_Buildroot_CN.pdf》开发指南，可在`docs/cn/Linux/System`目录下获取，并会不断完善更新。

4.3 Debian开发指南

为帮助开发工程师更快上手熟悉Debian系统的开发调试，随 SDK 发布《Rockchip_Developer_Guide_Debian_CN.pdf》开发指南，可在`docs/cn/Linux/System`目录下获取，并会不断完善更新。

4.4 第三方OS移植

为帮助开发工程师更快上手熟悉第三方OS的移植适配，随 SDK 发布《Rockchip_Developer_Guide_Third_Party_System_Adaptation_CN.pdf》开发向导，可在`docs/cn/Linux/System`目录下获取，并会不断完善更新。

4.5 DPDK开发指南

为帮助开发工程师更快上手熟悉 DPDK的开发调试，随 SDK 发布
《Rockchip_Developer_Guide_Linux_DPDK_CN.pdf》和
《Rockchip_Developer_Guide_Linux_GMAC_DPDK_CN.pdf》
开发向导，分别可在`docs/cn/Linux/DPDK`和`docs/cn/Common/GMAC`目录下获取，并会不断完善更新。

4.6 实时性Linux开发说明

随着产品对实时性能要求的提高，标准Linux的实时性已经满足不了很多产品的需求，需要对标准Linux进行一定的优化，来提高实时性能，比如：PREEMPT_RT，Xenomai实时系统。

下面是在 Buildroot 上分别使用PREEMPT_RT，Xenomai进行压力测试的延迟情况，以RK3572为例，具体如下：

压力测试：

```
stress-ng -c 8 --io 2 --vm 2 --vm-bytes 512M --timeout 1000000s
```

- PREEMPT_RT

```
root@rk3572-buildroot:/# cyclictest -c 0 -m -t 8 -a 0-7 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 13.67 13.67 13.60 14/325 5269

T: 0 ( 2552) P:99 I:1000 C:86399989 Min:     3 Act:    4 Avg:    6 Max:     103
T: 1 ( 2553) P:99 I:1500 C:57599990 Min:     3 Act:    7 Avg:    9 Max:     114
T: 2 ( 2554) P:99 I:2000 C:43199988 Min:     3 Act:    6 Avg:    8 Max:     108
T: 3 ( 2555) P:99 I:2500 C:34559987 Min:     3 Act:    6 Avg:    8 Max:     126
T: 4 ( 2556) P:99 I:3000 C:28799987 Min:     3 Act:    9 Avg:   11 Max:      79
T: 5 ( 2557) P:99 I:3500 C:24685700 Min:     3 Act:    5 Avg:   11 Max:      88
T: 6 ( 2558) P:99 I:4000 C:21599985 Min:     4 Act:   10 Avg:   11 Max:      98
T: 7 ( 2559) P:99 I:4500 C:19199984 Min:     4 Act:   12 Avg:   10 Max:     105
（测试24小时）

隔离CPU4~5(kernel cmdline加上"isolcpus=4-5 nohz_full=4-5"):
root@rk3572-buildroot:/# cyclictest -c 0 -m -t 2 -a 4-5 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 14.13 14.17 14.08 16/315 6997

T: 0 ( 3563) P:99 I:1000 C:86399998 Min:     3 Act:    3 Avg:    3 Max:      27
T: 1 ( 3564) P:99 I:1500 C:57599995 Min:     3 Act:    3 Avg:    3 Max:      19
（测试24小时）
```

- Xenomai Cobalt Mode

```
root@rk3572-buildroot:/# echo 0 > /proc/xenomai/latency
root@rk3572-buildroot:/# /usr/demo/cyclictest -c 0 -m -t 8 -a 0-7 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 13.09 13.12 13.09 13/261 7378
```

```
T: 0 ( 2547) P:99 I:1000 C:86399996 Min:      2 Act:    6 Avg:    6 Max:     107
T: 1 ( 2548) P:99 I:1500 C:57599997 Min:      2 Act:    5 Avg:    8 Max:     109
T: 2 ( 2549) P:99 I:2000 C:43199997 Min:      2 Act:    4 Avg:    9 Max:     123
T: 3 ( 2550) P:99 I:2500 C:34559996 Min:      2 Act:    4 Avg:    9 Max:     119
T: 4 ( 2551) P:99 I:3000 C:28799997 Min:      2 Act:    9 Avg:   14 Max:     113
T: 5 ( 2552) P:99 I:3500 C:24685711 Min:      3 Act:   11 Avg:   13 Max:     105
T: 6 ( 2553) P:99 I:4000 C:21599996 Min:      3 Act:   10 Avg:    9 Max:      91
T: 7 ( 2554) P:99 I:4500 C:19199996 Min:      4 Act:   13 Avg:   11 Max:     110
（测试24小时）

隔离CPU4~5(kernel cmdline加上"isolcpus=4-5 nohz_full=4-5"):
root@rk3572-buildroot:/# echo 0 > /proc/xenomai/latency
root@rk3572-buildroot:/# /usr/demo/cyclictest -c 0 -m -t 2 -a 4-5 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 13.38 13.47 13.52 12/231 5898

T: 0 ( 2444) P:99 I:1000 C:86399996 Min:      2 Act:    3 Avg:    3 Max:      34
T: 1 ( 2445) P:99 I:1500 C:57599997 Min:      2 Act:    3 Avg:    3 Max:      30
（测试24小时）
```

优化说明：

1. RK3572的`ddr.bin`针对实时系统进行了专门优化。在uboot工程里面需要用`./make.sh rk3572-rt`生成loader.bin，可打包集成实时配置的 DDR初始化固件。
2. RK3572分为3个Cluster，每个Cluster的L2是相互独立的，建议隔离CPU4-5(Cluster1)或CPU6-7(Cluster2)作为实时核心，隔离的CPU会提高访问DDR的优先级。

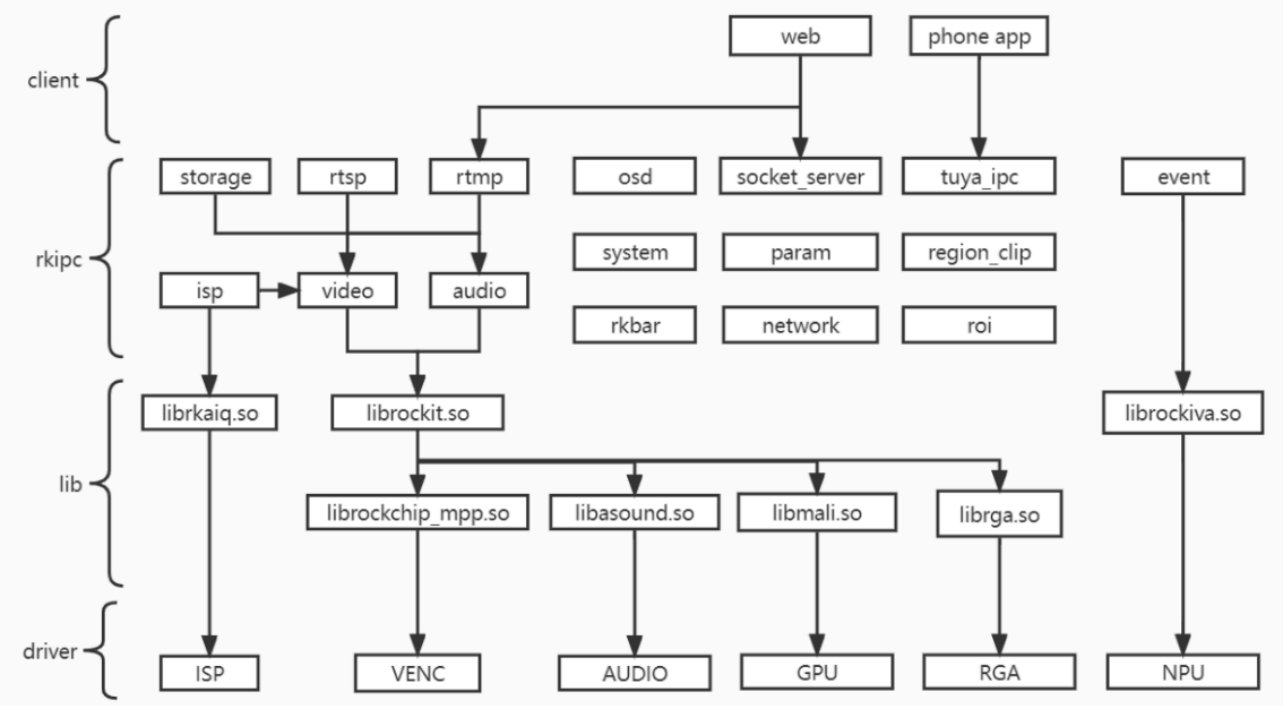
详情请参考`docs/Patches/Real-Time-Performance/`开发补丁包和说明。

Rockchip芯片在不同RT Linux实现下的实时性能测试数据参考如下文档：

`docs/cn/Common/PERF/Rockchip_Developer_Guide_Linux_RealTime_Performance_Test_Report_CN.pdf`

4.7 RKIPC开发指南

针对RK平台的IPC产品，支持网页和RTSP/RTMP预览，参数动态修改。RKIPC整体框架如下：



RKIPC应用开发指南详见如下文档：
`docs/cn/Linux/ApplicationNote/Rockchip_Developer_Guide_Linux_RKIPC_CN.pdf`。

4.8 Rockchip Linux updateEngine升级指南

Rockchip Linux 平台支持 Recovery 模式和 Linux A/B 模式两种 OTA 升级方案，均通过 updateEngine 工具完成，工具源码位于 `external/recovery/update_engine`。

升级方案	优点	缺点	参考配置
Recovery 模式	升级完整性有保障，异常掉电后可续传	须重启进入 Recovery 模式，不能在正常系统中直接升级	默认已开启，无需额外配置
Linux A/B 模式	无需重启进入升级模式，升级失败可自动回退	需两套系统分区，占用更多 Flash 空间	`device/rockchip/.chips/rk3572/rockchip_rk3572_evb1_v10_ab_defconfig`

设备端执行 OTA 升级时，除 SD/USB 启动盘自动升级流程外，均需通过 `--upgrade` 指定升级分区。`--upgrade=all` 不包含 `parameter` 和 `bootloader`，如需升级 GPT 或 IDBlock，需显式指定对应 item。量产后的 OTA 不建议默认升级 `parameter` 和 `bootloader`。

4.8.1 Recovery 模式 OTA 升级

在正常系统中执行以下命令触发升级流程。

网络升级：

```
updateEngine --image_url=http://<server_ip>/update.img \
  --misc=update \
  --savepath=/userdata/update.img \
  --upgrade=boot,recovery,rootfs \
  --reboot &
```

本地升级：

```
updateEngine --image_url=/userdata/update.img \
  --misc=update \
  --savepath=/userdata/update.img \
  --upgrade=boot,recovery,rootfs \
  --reboot &
```

升级流程：下载固件 → 升级 recovery 分区 → 重启进入 Recovery 模式 → 升级指定分区 → 重启回正常系统。

如果需要升级包中除 `userdata` 外的普通分区，可使用 `--upgrade=all,-userdata`。升级包保存在 `/userdata` 时，不应在同一次 OTA 中升级 `userdata`。

4.8.2 Linux A/B 模式 OTA 升级

执行以下命令生成 OTA 升级固件：

```
./build.sh ota-updateimg
```

网络升级：

```
updateEngine --image_url=http://<server_ip>/update-ota.img \
  --update \
  --upgrade=boot,system \
  --reboot
```

本地升级：

```
updateEngine --image_url=/userdata/update-ota.img \
  --update \
  --upgrade=boot,system \
  --reboot
```

升级流程：下载固件 → 升级备用 slot 分区 → 设置备用 slot 为下次引导分区 → 重启切换。

A/B 系统下 rootfs 分区必须命名为 `system_a/system_b`，升级命令使用 `--upgrade=boot,system`，updateEngine 会根据当前 slot 自动选择非当前 slot 中的目标分区。

4.8.3 差分升级

SDK 提供差分升级工具 `tools/linux/Linux_Diff_Firmware/`，支持 U-Boot、trust、kernel 及 squashfs rootfs 分区差分，可显著减少升级包体积。差分升级尚不支持 ext2/ext4 文件系统、加密分区和 A/B 模式。

```
sudo apt install bsdiff
pip install ubi_reader
cd tools/linux/Linux_Diff_Firmware/
./mk-diff-ota.sh update_old.img update_new.img update_diff.img
```

生成的 `update_diff.img` 可按普通 OTA 包方式使用。旧固件需与目标设备当前固件保持一致，且目标设备空闲分区空间需满足差分包和差分应用过程中生成的新镜像缓存需求。

4.8.4 SD 卡启动盘升级

使用 `tools/windows/SDDiskTool` 工具制作 SD 卡启动盘，插入设备上电后自动执行升级。Recovery 模式使用 `update.img` 制卡。

> **说明：** A/B 模式 SD 卡升级需在 defconfig 中配置 `RK_SDCARD_UPDATE=y`，并确保 `recovery.img` 已编译生成。执行 `./build.sh updateimg` 后，SDK 将生成 `update-ab.img` 和 `update-sd.img`。使用 `update-sd.img` 制作 SD 卡启动盘后，需将 `update-ab.img` 复制到 SD 卡根目录并重命名为 `sdupdate.img`。

Rockchip Linux updateEngine 升级详细说明请参考：

`docs/cn/Linux/Recovery/Rockchip_Developer_Guide_Linux_Upgrade_CN.pdf`

4.9 Rockchip Linux ROS2 软件开发指南

Rockchip Linux SDK 基于 Buildroot，支持从源码交叉编译 ROS2，当前支持 Jazzy 和 Humble 两个 LTS 版本，推荐使用 Jazzy。ROS2 在独立的 Docker 环境中完成交叉编译，与 SDK 解耦。编译脚本、Dockerfile 及补丁统一位于 SDK 的 `docs/Patches/ROS2/` 目录。

4.9.1 编译 ROS2 依赖包

在 Buildroot defconfig 中添加 ROS2 依赖配置，重新编译 rootfs：

```
# 以 Jazzy 为例，在 buildroot/configs/rockchip_rk3572_defconfig 末尾追加
#include "ros2/jazzy_dep.config"

./build.sh rootfs
```

4.9.2 准备 Docker 环境

安装 Docker 及必要工具：

```
sudo apt install docker.io python3-vcstool qemu-user-binfmt
sudo usermod -aG docker $USER && newgrp docker
```

下载 ROS2 源码并应用 SDK 补丁后，使用统一 Dockerfile 构建镜像，通过 `--build-arg ROS_DISTRO` 指定版本：

```
docker build --build-arg ROS_DISTRO=jazzy \
  --build-arg HOST_USER=$(whoami) --build-arg HOST_UID=$(id -u) --build-arg HOST_GID=$(id -g) \
  -t jazzy-ros2-builder -f rosdep.Dockerfile ./
```

4.9.3 在 Docker 中交叉编译 ROS2

将 Buildroot output 目录挂载至 Docker，执行 ROS2 交叉编译，产物输出至 `target/opt/ros/`，完整编译约需 1~2 小时：

```
docker run -it --rm --privileged \
  -u $(id -u):$(id -g) \
  -v /etc/passwd:/etc/passwd:ro -v /etc/group:/etc/group:ro \
  --mount type=bind,source=<SDK>/buildroot/output/rockchip_rk3572/rockchip_rk3572,target=/buildroot \
  jazzy-ros2-builder:latest

./build_ros2.sh jazzy
```

4.9.4 打包 rootfs 并验证

退出 Docker 后重新打包 rootfs，烧录后在板端执行验证：

```
./build.sh rootfs

source /opt/ros/local_setup.sh
ros2 run demo_nodes_cpp talker &
ros2 run demo_nodes_cpp listener
```

> **说明：** 依赖 Qt/X11 的软件包，如 rviz、rqt、turtlesim 等，在 Buildroot 环境中不支持编译和运行。

ROS2 编译详细说明请参考：

`docs/cn/Linux/ApplicationNote/Rockchip_Instruction_Linux_ROS/Rockchip_Instruction_Linux_ROS2_CN.pdf`

4.10 Rockchip IDC 软件开发指南

IDC（Image Distortion Correction，图像畸变矫正）利用 GPU 并行计算能力对图像畸变进行高效矫正，当前依赖 rockit，面向 RKIPC 场景使用。以下以 rockchip_rk3572_evb1_v10 Buildroot 配置为例，说明 IDC Demo 的编译与运行方法。

4.10.1 编译配置

在 Buildroot 配置中启用以下选项：

```
BR2_PACKAGE_COMMON_ALGORITHM=y
BR2_PACKAGE_SAMPLES=y
BR2_PACKAGE_SAMPLES_RK3572=y
```

在已完成 Buildroot 编译的基础上，依次执行以下命令重新编译相关模块：

```
make rockit-recreate
make common_algorithm-recreate
make samples-recreate
```

4.10.2 运行示例

板端运行示例命令可参考 `external/samples/simple_test/simple_cmd_rk3572.txt`，典型用法如下：

```
simple_vi_gdc_vo -w 1920 -h 1088 -m 1 -M 3 -W 1 \
  -s /data/rkalg_idc_gpu_meshxy_step16x8.bin \
  -f /usr/lib/libmali.so -l 100
```

> **说明：** 运行前需确认 `/data/rkalg_idc_gpu_meshxy_step16x8.bin` 文件存在。该文件因摄像头型号而异，须自行标定生成。SDK 中提供了一份样例文件可用于基本验证，路径为 `external/common_algorithm/misc/rkidc_config/rkalg_idc_gpu_meshxy_step16x8.bin`。

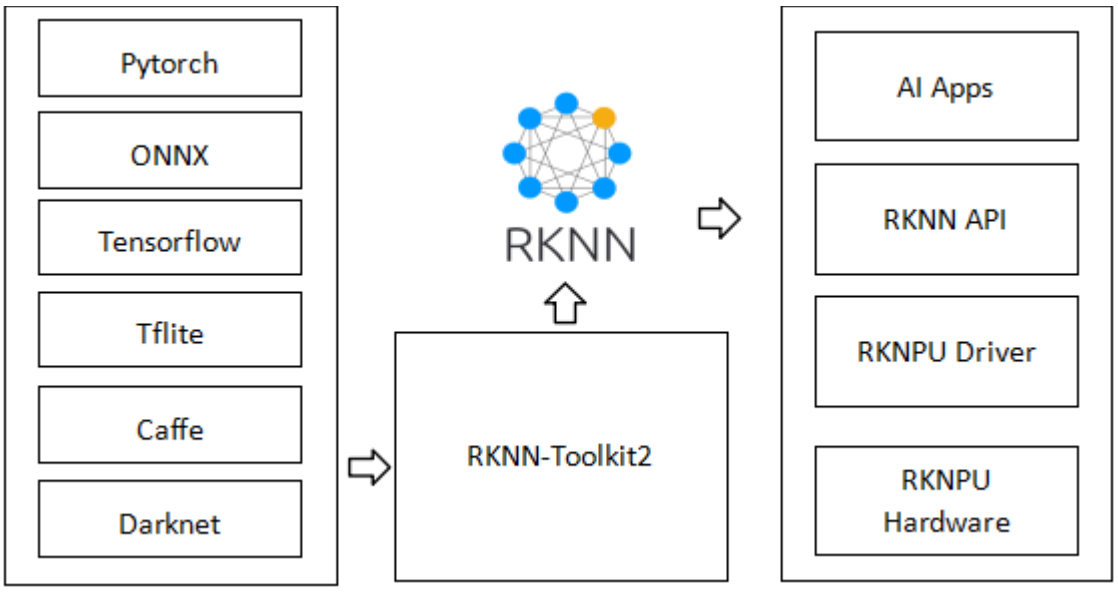
IDC 详细说明请参考：

`docs/cn/Common/Rockchip_IDC_Developer_Guide_CN.pdf`

`docs/cn/Common/Rockchip_IDC_Developer_Tutorial_CN.pdf`

4.11 RKNN 开发指南

RKNN 软件栈可帮助用户快速将 AI 模型部署至 Rockchip 芯片。整体框架如下：



使用 RKNPU 前，用户需先在计算机上运行 PC 端工具（RKNN-Toolkit2 或 RKNN3-Toolkit），将训练好的模型转换为 RKNN 格式模型，随后通过板端 Runtime 提供的 C/C++ 或 Python API 在开发板上进行推理。

不同芯片采用不同代次的 NPU 软件栈，请按下表选择对应平台的工具链与文档：

NPU 软件栈	适用芯片	PC 端工具	板端 Runtime	内核驱动
RKNPU2	RK3576、RK3588 等	RKNN-Toolkit2（`external/rknn-toolkit2`）	`external/rknpu2`	`drivers/rknpu`
RKNPU3	RK3572	RKNN3-Toolkit（`external/rknn3-toolkit`）	`external/rknn3-runtime`	`drivers/rknpu`

4.11.1 RKNPU2 平台（RK3576、RK3588 等）

RKNN-Toolkit2

RKNN-Toolkit2 是在 PC 上进行 RKNN 模型生成及性能评估的开发套件；RKNN-Toolkit-Lite2 为 Rockchip NPU 平台提供 Python 编程接口，协助用户部署 RKNN 模型并加速 AI 应用实现。开发套件位于 `external/rknn-toolkit2` 目录下，用于实现模型转换、优化、量化、推理、性能评估和精度分析等功能。

基本功能如下：

功能	说明
模型转换	支持 Pytorch / TensorFlow / TFLite / ONNX / Caffe / Darknet 的浮点模型 支持 Pytorch / TensorFlow / TFLite 的量化感知模型（QAT） 支持动态输入模型（动态化/原生动态） 支持大模型
模型优化	常量折叠 / OP 矫正 / OP Fuse&Convert / 权重稀疏化 / 模型剪枝
模型量化	支持量化类型：非对称 i8 / fp16 支持 Layer / Channel 量化方式；Normal / KL / MMSE 量化算法 支持混合量化以平衡性能和精度

功能	说明
模型推理	支持在 PC 上通过模拟器进行模型推理 支持将模型传到 NPU 硬件平台上完成模型推理（连板推理） 支持批量推理，支持多输入模型
模型评估	支持模型在 NPU 硬件平台上的性能和内存评估
精度分析	支持量化精度分析功能（模拟器 / NPU）
附加功能	支持版本 / 设备查询功能等

RKNN Runtime

RKNN Runtime 的开发说明在工程目录 `external/rknpu2` 下，用于推理 RKNN-Toolkit2 生成的 rknn 模型，为 Rockchip NPU 平台提供 C/C++ 编程接口。具体使用说明请参考 `external/rknpu2/doc/` 目录文档：

```
├── 01_Rockchip_RKNPU_Quick_Start_RKNN_SDK_V2.3.2_CN.pdf
├── 02_Rockchip_RKNPU_User_Guide_RKNN_SDK_V2.3.2_CN.pdf
├── 03_Rockchip_RKNPU_API_Reference_RKNN_Toolkit2_V2.3.2_CN.pdf
├── 04_Rockchip_RKNPU_API_Reference_RKNNRT_V2.3.2_CN.pdf
└── 05_RKNN_Compiler_Support_Operator_List_V2.3.2.pdf
```

RKNN Driver

RKNN 内核驱动负责与 NPU 硬件交互，已集成于 Rockchip 内核代码 `drivers/rknpu` 目录中。

RKNN LLM

RKNN LLM 是 Rockchip 面向大语言模型（Large Language Model, LLM）场景优化的推理框架，源码位于 `external/rknn-llm`，用于在 Rockchip NPU 硬件平台上高效部署和运行大规模预训练语言模型（如 LLaMA、ChatGLM、Qwen 等）。具体使用说明请参考 `external/rknn-llm/doc/` 目录文档：

```
└── Rockchip_RKLLM_SDK_CN_1.3.0.pdf
```

4.11.2 RKNPU3 平台（RK3572）

RKNN3-Toolkit

RKNN3 SDK 提供将 AI 模型部署到 RK3572 的完整软件栈。RKNN3-Toolkit 是 PC 端开发套件，位于 `external/rknn3-toolkit` 目录下，支持模型转换、推理和性能评估等。具体使用说明请参考 `external/rknn3-toolkit/doc/` 目录文档：

```
├── 01_Rockchip_RKNPU3_Quick_Start_RKNN3_SDK_V1.0.4_CN.pdf
├── 02_Rockchip_RKNPU3_User_Guide_RKNN3_SDK_V1.0.4_CN.pdf
├── 03_Rockchip_RKNPU3_API_Reference_RKNN3_Toolkit_V1.0.4_CN.pdf
├── 04_Rockchip_RKNPU3_API_Reference_RKNNRT3_V1.0.4_CN.pdf
└── 05_Rockchip_User_Guide_RKNN3_Toolkit_Lite_V1.0.4_CN.pdf
```

RKNN3 Runtime

RKNN3 Runtime 为板端运行时库，位于 `external/rknn3-runtime` 目录下，提供 C/C++ 编程接口，用于部署 RKNN3 模型并加速 AI 应用。

RKNN3 Driver

RKNN3 内核驱动同样集成于 Rockchip 内核代码 `drivers/rknpu` 目录中。

RKNN3 Model Zoo

RKNN3 Model Zoo 位于 `external/rknn3_model_zoo` 目录下，提供 CNN / LLM / VLM 等多种模型的转换与部署示例参考实现。

RKLLM3

面向 RK3572 的 LLM 推理服务位于 `external/rknn3-runtime/rkllm3-server`，配合 RKNN3 Model Zoo 完成大语言模型在 RKNPU3 平台上的部署。

4.12 RKAI 开发指南

RKAI 提供大语言模型（LLM）和视觉语言模型（VLM）验证程序及应用开发接口，源码位于 `app/rkai`。RKAI 同时支持 RKNN3（RKNPU3）与 RKLLM/RKNPU2 两种后端，NPU 软件栈的选择与对应芯片平台请参考前述「RKNN 开发指南」。以下以适用于 RK3572 的 RKNN3 后端为例；其他 RKNPU2 平台按 RKLLM/RKNPU2 后端方式使用，具体启用配置以 Linux6.12 SDK 中对应平台配置为准。

4.12.1 编译配置

在 Buildroot 配置中启用 RKAI，重新编译 rootfs：

```
#include "ai/rkai.config"
./build.sh rootfs
```

4.12.2 模型文件

适配 RKNN3 的 LLM 模型需要 `.rknn` 模型文件，以及配套的 `.weight`、`.tokenizer.gguf` 和 `.embed.bin` 文件。VLM 模型还需要配套的 Vision `.rknn` 和 `.weight` 文件。

RKLLM/RKNPU2 后端的 LLM 使用 `.rkllm` 模型，VLM 使用 Vision `.rknn` 模型和 LLM `.rkllm` 模型，配置文件为 `rkai_cfg.ini`。

RKNN3 后端的模型转换及部署示例请参考：

- RKNN3-Toolkit：https://github.com/airockchip/rknn3-toolkit
- RKNN3 Model Zoo：https://github.com/airockchip/rknn3-model-zoo

RKLLM/RKNPU2 后端的模型转换工具请参考：

- LLM 模型转换使用 RKLLM-Toolkit：https://github.com/airockchip/rknn-llm
- Vision 模型转换使用 RKNN-Toolkit2：https://github.com/airockchip/rknn-toolkit2

4.12.3 LLM 运行示例

RKNN3 后端：

```
rkai_LLM_Demo \
  --llm_model /userdata/<llm_model_dir>/ \
  --config /usr/share/app_conf/config/rkai_cfg_rk3572.ini
```

RKLLM/RKNPU2 后端：

```
rkai_LLM_Demo \
  --llm_model /userdata/<llm_model>.rkllm \
  --config /usr/share/app_conf/config/rkai_cfg.ini
```

4.12.4 VLM 运行示例

RKNN3 后端：

```
rkai_VLM_Demo \
  --vision_model /userdata/<vlm_model_dir>/ \
  --llm_model /userdata/<vlm_model_dir>/ \
  --image_path /userdata/<vlm_model_dir>/<input_image>.jpg \
  --config /usr/share/app_conf/config/rkai_cfg_rk3572.ini
```

RKLLM/RKNPU2 后端：

```
rkai_VLM_Demo \
  --vision_model /userdata/<vision_model>.rknn \
  --llm_model /userdata/<llm_model>.rkllm \
  --image_path /userdata/<input_image>.jpg \
  --config /usr/share/app_conf/config/rkai_cfg.ini
```

RKAI 详细说明请参考：

`docs/cn/Linux/RKAI/Rockchip_Developer_Guide_Linux_RKAI_CN.pdf`

4.13 软件更新记录

- 软件发布版本升级可通过工程 XML 进行查看，具体方法如下：

```
.repo/manifests$ realpath rockchip_linux6.12_release.xml
```

例如：打印的版本号为 v1.0.0，更新时间为 20260620，对应路径如下：

```
<SDK>/.repo/manifests/release/rockchip_linux6.12_release_v1.0.0_20260620.xml
```

- 软件发布版本升级更新内容通过工程文本可以查看，参考工程目录：

```
<SDK>/.repo/manifests/rockchip_linux6.12_SDK_Note.md
或者
<SDK>/docs/cn/Release/Linux6.12/rockchip_linux6.12_SDK_Note.md
```

- 板端可通过如下方式获取版本信息

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-6.12-stan-rkr1
ID=buildroot
VERSION_ID=2026.02.2
PRETTY_NAME="Buildroot 2026.02.2"
ID_LIKE="buildroot"
RK_BUILD_INFO="<build_user> Sat Jun 20 14:35:23 CST 2026 - rockchip_<chip>"
```

5. SDK 工程目录介绍

SDK 工程目录包含有 buildroot、debian、yocto、app、kernel、u-boot、device、docs、external、prebuilts等目录。采用manifest来管理仓库，用repo工具来管理每个目录或其子目录会对应的 git 工程。

- app：存放上层应用 APP，主要是一些应用Demo。
- buildroot：基于 Buildroot（2026.02）开发的根文件系统。
- debian：基于 Debian trixie(13) 开发的根文件系统。
- device/rockchip：存放芯片板级配置以及SDK编译和打包固件的脚本和文件等。
- docs：存放通用开发指导文档、芯片平台相关文档、Linux 系统开发相关文档、其他参考文档等。
- external：存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等。
- kernel：存放 Kernel 开发的代码。
- output：存放每次生成的固件信息、编译信息、XML、主机环境等。
- prebuilts：存放交叉编译工具链。
- rkbin：存放 Rockchip 相关二进制和工具。
- rockdev：存放编译输出固件，实际软链接到`output/firmware`。
- tools：存放 Linux 和 Windows 操作系统下常用工具。
- u-boot：存放基于 v5 版本进行开发的 U-Boot 代码。
- yocto：基于 Yocto 6.0 开发的根文件系统。

6. SDK交叉编译工具链介绍

鉴于Rockchip Linux SDK目前只在Linux PC环境下编译，我们也仅提供了Linux下的交叉编译工具链。prebuilts目录下的预置工具链用于U-Boot和Kernel编译。具体Rootfs需要使用各自对应的工具链，或者使用第三方工具链进行编译。（以下例子以 RK3572 为例）

6.1 U-Boot 及Kernel编译工具链

SDK prebuilts 目录预置的交叉编译工具链目前用于 U-Boot 及 Kernel 编译，如下：

目录	说明
prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-14.3.rel1-x86_64-aarch64-none-linux-gnu	gcc arm 14.3.rel1 64位工具链
prebuilts/gcc/linux-x86/arm/arm-gnu-toolchain-14.3.rel1-x86_64-arm-none-linux-gnueabihf	gcc arm 14.3.rel1 32位工具链

可从以下地址下载工具链：

点击这里

6.2 Buildroot工具链

6.2.1 配置编译环境

若需要编译单个模块或者第三方应用，需交叉编译环境进行配置。比如RK3572其交叉编译工具位于 `buildroot/output/<RK_DEFCONFIG>/rockchip_rk3572/host/usr` 目录下，其中 `<RK_DEFCONFIG>` 表示当前选择的 Buildroot defconfig 目录名。需要将工具的bin/目录和 `aarch64-buildroot-linux-gnu/bin/` 目录设为环境变量，在顶层目录执行自动配置环境变量的脚本：

```
source buildroot/envsetup.sh rockchip_rk3572
```

输入命令查看：

```
cd buildroot/output/<RK_DEFCONFIG>/rockchip_rk3572/host/usr/bin
./aarch64-linux-gcc --version
```

此时会打印如下信息：

```
aarch64-linux-gcc.br_real (Buildroot -gf133d0a00d) 14.3.0
```

6.2.2 打包工具链

Buildroot 支持将内置工具链打包为压缩包，以供第三方应用单独编译使用。有关如何打包工具链的详细信息，请参阅 Buildroot 官方文档：

```
buildroot/docs/manual/using-buildroot-toolchain.adoc
```

在 SDK 中，可以直接运行以下命令来生成工具链包：

```
./build.sh bmake:sdk
```

生成的工具链包位于 buildroot/output/*/images/ 目录，名为 aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz，供有需求的用户使用。解压后，gcc 的路径将是：

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

6.3 Debian工具链

使用 Docker 环境、gcc 或者 dpkg-buildpackage 进行相关编译。

6.4 Yocto工具链

参考如下：

- Building your own recipes from first principles
- New Recipe

7. SDK编译说明

SDK可通过 make 或 ./build.sh 加目标参数进行相关功能的配置和编译。
具体参考 device/rockchip/common/README.md 编译说明。

为了确保SDK每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 ./build.sh cleanall。

（以下例子以 RK3572 为例）

一键编译

芯片	类型	参考机型	一键编译	rootfs编译	kernel编译	uboot编译
RK3572	开发板	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_evb1_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	开发板（ab分区升级）	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_evb1_v10_ab_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	开发板	RK3572 EVB2	./build.sh lunch:rockchip_rk3572_evb2_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb2-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	单目IPC	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_ipc_evb1_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new

芯片	类型	参考机型	一键编译	rootfs编译	kernel编译	uboot编译
RK3572	机器人	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_evb1_robot_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	单目UVC	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_uvc_evb1_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new

> **注意：**
>
> - 上表以 RK3572 为例，其他芯片将命令中的芯片名、defconfig、DTS 替换为对应芯片即可，即 `./build.sh lunch:rockchip_<chip>_<board>_defconfig`、`<chip>.config`、`<chip>-<board>-linux.img`、`./make.sh <chip>`。
> - **Rootfs编译**：
>   默认是编译Buildroot系统，如果需要其他系统，设置相应的环境变量即可。比如
>   编译buildroot系统：RK_ROOTFS_SYSTEM=buildroot ./build.sh
>   编译debian系统：RK_ROOTFS_SYSTEM=debian ./build.sh
>   编译yocto系统：RK_ROOTFS_SYSTEM=yocto ./build.sh
> - **Kernel、U-boot编译**：其中需要指定工具链。
> - 比如SDK内置32位工具链：
>   export CROSS_COMPILE=../prebuilts/gcc/linux-x86/arm/arm-gnu-toolchain-14.3.rel1-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
> - 比如SDK内置64位工具链：
>   export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-14.3.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-

7.1 SDK编译命令查看

`make help`，例如：

```
$ make help
  menuconfig          - interactive curses-based configurator
  oldconfig           - resolve any unresolved symbols in .config
  syncconfig          - Same as oldconfig, but quietly, additionally update deps
  olddefconfig        - Same as syncconfig but sets new symbols to their default value
  savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
  ...
```

make实际运行是`./build.sh`

即也可运行`./build.sh <target>`来编译相关功能，具体可通过`./build.sh help`查看具体编译命令。

```
./build.sh -h

############### Rockchip Linux SDK ###############

Manifest: rockchip_linux6.12.xml
GIT commit: "e36cd901 tools: Support retaining original rootfs tools"
```

```
Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]              choose chip
defconfig[:<config>]                  choose defconfig
 *_defconfig                          switch to specified defconfig
    available defconfigs:
    rockchip_defconfig
    rockchip_rk3572_evb1_robot_defconfig
    rockchip_rk3572_evb1_v10_ab_defconfig
    rockchip_rk3572_evb1_v10_defconfig
    rockchip_rk3572_evb2_v10_defconfig
    rockchip_rk3572_evb3_v10_defconfig
    rockchip_rk3572_iotest_v10_defconfig
    rockchip_rk3572_ipc_evb1_v10_defconfig
    rockchip_rk3572_test1_v10_defconfig
    rockchip_rk3572_test2_v10_defconfig
    rockchip_rk3572_test3_v10_defconfig
    rockchip_rk3572_uvc_evb1_v10_defconfig
 olddefconfig                         resolve any unresolved symbols in .config
 savedefconfig                        save current config to defconfig
 menuconfig                           interactive curses-based configurator
config                                modify SDK defconfig
config-rootfs-type                    modify SDK rootfs type config
config-usb-gadget                     modify SDK USB gadget config
config-wifibt                         modify SDK Wi-Fi/BT config
config-rootfs-overlay                 modify SDK rootfs overlay config
config-overlay                        alias of config-rootfs-overlay
config-post-rootfs                    modify SDK post rootfs config
config-extra-parts                    modify SDK extra partitions config
print-parts                           print partitions
list-parts                            alias of print-parts
edit-parts                            edit raw partitions
misc                                  pack misc image
kernel-5.10[:dry-run]                 build kernel 5.10
kernel-6.1[:dry-run]                  build kernel 6.1
kernel-6.12[:dry-run]                 build kernel 6.12
kernel[:dry-run]                      build kernel
recovery-kernel[:dry-run]             build kernel for recovery
kernel-modules[:<dst dir>:dry-run]    build kernel modules
modules[:<dst dir>:dry-run]           alias of kernel-modules
linux-headers[:<arch>:dry-run]        build linux-headers
kernel-config[:dry-run]               modify kernel defconfig
kconfig[:dry-run]                     alias of kernel-config
kernel-make[:<arg1>:<arg2>]           run kernel make
kmake[:<arg1>:<arg2>]                 alias of kernel-make
wifibt[:<dst dir>[:<chip>]]           build Wifi/BT
amp                                   build and pack amp system
buildroot-config[:<config>]           modify buildroot defconfig
bconfig[:<config>]                    alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]        run buildroot make
bmake[:<arg1>:<arg2>]                 alias of buildroot-make
buildroot-sdk                         build the buildroot SDK tarball
```

```
bsdk                            alias of buildroot-sdk
edit-debian-packages                edit debian package list
rootfs[:<rootfs type>]              build default rootfs
buildroot                       build buildroot rootfs
yocto                           build yocto rootfs
debian                          build debian rootfs
recovery                        build recovery
security-createkeys                 create keys for security
security-misc                       build misc with system encryption key
security-ramboot[:system_image]         build security ramboot
security-system[:system_image]          build security system
security-remote-sign                build remote signed image
loader[:dry-run]                    build loader (u-boot)
uboot[:dry-run]                     build u-boot
u-boot[:dry-run]                    alias of uboot
extra-parts                     pack extra partition images
firmware                        pack and check firmwares
edit-package-file                   edit package-file
edit-ota-package-file               edit package-file for OTA
updateimg                       build update image
ota-updateimg                       build update image for OTA
sd-updateimg                        build update image for SDcard
all                             build images
release[:<subdir>[:<name>]]         release images and build info
all-release[:<subdir>[:<name>]]         build and release images
shell                           setup a shell for developing
buildroot-shell                     setup a shell for buildroot developing
bshell                          alias of buildroot-shell
yocto-shell                         setup a shell for yocto developing
yshell                          alias of yocto-shell
cleanall                        cleanup all
clean-misc                          cleanup misc
clean-config                        cleanup config
clean-recovery                      cleanup recovery
clean-amp                           cleanup amp
clean-kernel                        cleanup kernel
clean-rootfs                        cleanup rootfs
clean-firmware                      cleanup firmware
clean-security                      cleanup security
clean-extra-parts                   cleanup extra-parts
clean-updateimg                     cleanup updateimg
clean-loader                        cleanup loader
post-rootfs <rootfs dir>                trigger post-rootfs hook scripts
help                            display this information

Default option is 'all'.
```

7.2 SDK板级配置

Linux6.12 为覆盖 RK3572、RK3576、RK3588 等芯片的统一 SDK。编译前需先选择目标芯片，再选择该芯片下对应的板级配置（defconfig）。芯片选择与板级配置切换分别使用 `./build.sh chip` 与 `./build.sh lunch`，两者现象不同，下面分别说明。

7.2.1 芯片选择

`./build.sh chip` 用于选择目标芯片。执行后先列出 SDK 支持的芯片供选择（Pick a chip），选定芯片后会接着列出该芯片下的板级配置供选择（Pick a defconfig）：

```
./build.sh chip

################ Rockchip Linux SDK ###############

Pick a chip:

1. rk3572
2. rk3576
3. rk3588
Which would you like? [1]:
1
Switching to chip: rk3572

Pick a defconfig:

1. rockchip_defconfig
2. rockchip_rk3572_evb1_robot_defconfig
3. rockchip_rk3572_evb1_v10_ab_defconfig
4. rockchip_rk3572_evb1_v10_defconfig
5. rockchip_rk3572_evb2_v10_defconfig
6. rockchip_rk3572_evb3_v10_defconfig
7. rockchip_rk3572_iotest_v10_defconfig
8. rockchip_rk3572_ipc_evb1_v10_defconfig
9. rockchip_rk3572_test1_v10_defconfig
10. rockchip_rk3572_test2_v10_defconfig
11. rockchip_rk3572_test3_v10_defconfig
12. rockchip_rk3572_uvc_evb1_v10_defconfig
Which would you like? [1]:
```

上述 Pick a defconfig 列出的是所选芯片的板级配置，此处以选择 rk3572 为例。

7.2.2 板级配置选择

`./build.sh lunch`（或 `make lunch`）用于在**已选定芯片**下切换板级配置，只列出当前芯片的板级配置（Pick a defconfig），不再选择芯片：

```
./build.sh lunch

################ Rockchip Linux SDK ###############

Pick a defconfig:

1. rockchip_defconfig
2. rockchip_rk3572_evb1_robot_defconfig
3. rockchip_rk3572_evb1_v10_ab_defconfig
4. rockchip_rk3572_evb1_v10_defconfig
```

```
5. rockchip_rk3572_evb2_v10_defconfig
6. rockchip_rk3572_evb3_v10_defconfig
7. rockchip_rk3572_iotest_v10_defconfig
8. rockchip_rk3572_ipc_evb1_v10_defconfig
9. rockchip_rk3572_test1_v10_defconfig
10. rockchip_rk3572_test2_v10_defconfig
11. rockchip_rk3572_test3_v10_defconfig
12. rockchip_rk3572_uvc_evb1_v10_defconfig
Which would you like? [1]:
```

下面以 RK3572 为例，其板级配置位于`<SDK>/device/rockchip/.chips/rk3572`目录：

板级配置	说明
rockchip_defconfig	默认配置，具体会软链接到RK3572 EVB1开发板配置
rockchip_rk3572_evb1_robot_defconfig	适用于RK3572 EVB1 机器人开发板配置
rockchip_rk3572_evb1_v10_ab_defconfig	适用于RK3572需要ab升级 开发板配置
rockchip_rk3572_evb1_v10_defconfig	适用于RK3572 EVB1 AIoT开发板配置
rockchip_rk3572_evb2_v10_defconfig	适用于RK3572 EVB2 AIoT开发板配置
rockchip_rk3572_evb3_v10_defconfig	适用于RK3572 EVB3 开发板配置
rockchip_rk3572_iotest_v10_defconfig	适用于RK3572 IO 测试开发板配置
rockchip_rk3572_ipc_evb1_v10_defconfig	适用于RK3572 单目IPC EVB1开发板配置
rockchip_rk3572_test1_v10_defconfig	适用于RK3572 test1 测试开发板配置
rockchip_rk3572_test2_v10_defconfig	适用于RK3572 test2 测试开发板配置
rockchip_rk3572_test3_v10_defconfig	适用于RK3572 test3 测试开发板配置
rockchip_rk3572_uvc_evb1_v10_defconfig	适用于RK3572 单目UVC EVB1开发板配置

> 大部分客户拿到的都是EVB1板子，适用通用的AIoT应用场景。

RK3576、RK3588 的板级配置流程与此相同，对应 defconfig 分别位于`<SDK>/device/rockchip/.chips/rk3576`、`<SDK>/device/rockchip/.chips/rk3588`目录，命名规则一致（`rockchip_<芯片>_<板型>_defconfig`），可通过`./build.sh chip`选择对应芯片后查看与选用。

其他功能的配置可通过`make menuconfig`来配置相关属性。

7.3 SDK定制化配置

SDK可通过`make menuconfig`进行相关配置，目前可配组件主要如下：

```
(rockchip_rk3572_evb1_v10_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto)  --->
[*] Loader (U-Boot)  --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image)  --->
    Boot (Android-style boot image)  --->
[*] Recovery (based on Buildroot)  --->
[ ] Security feature (secureboot, encryption, verity, etc.)
    Extra partitions (oem, userdata, etc.)  --->
    Firmware (partition table, misc image, etc.)  --->
[*] Update (Rockchip update image)  --->
    *** Fastboot is not supported for this chip ***
    Others configurations  --->
```

- Rootfs: 这里的Rootfs代表“根文件系统”，在这里可选择Buildroot、Yocto、Debian等不同的根文件系统配置。
- Loader (U-Boot): 这是引导加载器的配置，通常是U-Boot，用于初始化硬件并加载主操作系统。
- AMP: 多核异构启动方案，适用于需要实时性能的应用场景。
- Kernel: 这里配置内核选项，定制适合自己的硬件和应用需求的Linux内核。
- Boot: 这里配置Boot分区支持格式。
- Recovery (based on buildroot): 这是基于buildroot的recovery环境的配置，用于系统恢复和升级。
- Security: 安全功能开启，包含Secureboot开启方式、Optee存储方式、烧写Key等。
- Extra partitions: 用于配置额外的分区。
- Firmware: 在这里配置固件相关选项。
- Update (Rockchip update image): 用于配置Rockchip完整固件的选项。
- Others configurations: 其他额外的配置选项。

通过`make menuconfig`，可以提供一个基于文本的用户界面来选择和配置各种选项。
配置完成后，使用`make savedefconfig`命令保存这些配置，这样定制化编译就会根据这些设置来进行。

通过以上config，可选择不同Rootfs/Loader/Kernel等配置，进行各种定制化编译，这样就可以灵活地选择和配置系统组件以满足特定的需求。

7.4 全自动编译

进入工程根目录执行以下命令自动完成所有的编译：

```
./build.sh all # 编译模块并将基础固件打包到 output/firmware
./build.sh    # 默认目标为 all，作用与上一条命令相同

./build.sh updateimg # 基于基础固件生成 update.img 升级包
```

默认是 Buildroot，可以通过设置环境变量`RK_ROOTFS_SYSTEM`指定不同 rootfs。`RK_ROOTFS_SYSTEM`目前可设定三种系统：buildroot、debian、yocto。

比如需要 debian 可以通过以下命令进行生成：

```
export RK_ROOTFS_SYSTEM=debian
./build.sh
或
RK_ROOTFS_SYSTEM=debian ./build.sh
```

7.5 各模块编译

7.5.1 U-Boot编译

```
./build.sh uboot
```

7.5.2 Kernel编译

- 方法一

```
./build.sh kernel
```

- 方法二

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-14.3.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rk3572.config
make ARCH=arm64 rk3572-evb1-v10-linux.img -j4
```

> **说明：** `rk3572-evb1-v10-linux` 可替换为具体硬件对应的 DTS 名称，`-j` 后的并行数可根据主机 CPU 核数调整。

7.5.3 Recovery编译

进入工程根目录执行以下命令自动完成 Recovery 的编译及打包。

```
<SDK># ./build.sh recovery
```

编译后在 Buildroot 目录 `output/<RK_DEFCONFIG>/rockchip_rk3572_recovery/images` 生成 recovery.img。

> 注：recovery.img 是包含内核的镜像，所以每次 Kernel 更改，Recovery 是需要重新打包生成。Recovery 重新打包的方法如下：

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
buildroot# make recovery-reconfigure
buildroot# cd -
<SDK># ./build.sh recovery
```

> 注：Recovery是非必需的功能，有些板级配置不会设置。

7.5.4 Buildroot编译

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh rootfs
```

编译后在Buildroot目录`output/<RK_DEFCONFIG>/rockchip_rk3572/images`下生成不同格式的镜像，默认使用rootfs.ext4格式。

具体可参考Buildroot开发文档：

`<SDK>/docs/cn/Linux/System/Rockchip_Developer_Guide_Buildroot_CN.pdf`

7.5.4.1 Buildroot模块编译

可通过 `source buildroot/envsetup.sh` 来设置不同芯片和目标功能的配置

```
$ source buildroot/envsetup.sh
Top of tree: RK3572

You're building on Linux
Lunch menu...pick a combo:

78. rockchip_rk3572
79. rockchip_rk3572_ab
80. rockchip_rk3572_ipc
81. rockchip_rk3572_pcba
82. rockchip_rk3572_ramboot
83. rockchip_rk3572_recovery
84. rockchip_rk3572_robot
85. rockchip_rk3572_uvc
...

Which would you like? [1]:
```

默认选择78，`rockchip_rk3572`。然后进入RK3572的Buildroot目录，开始相关模块的编译。

其中`rockchip_rk3572_recovery`是用来编译Recovery模块。

比如编译 rockchip-test 模块，常用相关编译命令如下：

进入 buildroot目录

```
<SDK># cd buildroot
```

- 删除并重新编译 rockchip-test

```
buildroot# make rockchip-test-recreate
```

- 重编 rockchip-test

```
buildroot# make rockchip-test-rebuild
```

- 删除 rockchip-test

```
buildroot# make rockchip-test-dirclean
或者
buildroot# rm -rf output/rockchip_rk3572/build/rockchip-test-master/
```

7.5.5 Debian编译

```
./build.sh debian
```

编译后在 debian 目录下生成 rockchip-rootfs.img。

> **说明：** 需要预先安装相关依赖包

```
sudo apt-get install binfmt-support qemu-user-static live-build
sudo dpkg -i ubuntu-build-service/packages/*
sudo apt-get install -f
```

具体可参考Debian开发文档：

`<SDK>/docs/cn/Linux/System/Rockchip_Developer_Guide_Debian_CN.pdf`

7.5.6 Yocto 编译

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh yocto
```

编译后在 yocto 目录 build/latest 下生成 rootfs.img。

默认用户名登录是 root。Yocto 更多信息请参考 Rockchip Wiki。

FAQ：

- 上面编译如果遇到如下问题情况：

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).
Python can't change the filesystem locale after loading so we need a UTF-8
when Python starts or things won't work.
```

解决方法：

```
locale-gen en_US.UTF-8
export LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8
```

或者参考 setup-locale-python3

- 如果遇到git权限问题，导致编译出错。

由于git新版本增加了CVE-2022-39253安全检测补丁，而如果旧版本的Yocto就需要poky包含如下才可修复：

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba
Author: Robert Yang <liezhi.yang@windriver.com>
Date:   Fri Mar 24 00:09:02 2023 -0700

    bitbake: fetch/git: Fix local clone url to make it work with repo

    The "git clone /path/to/git/objects_symlink" couldn't work after the following
    change:

    https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56
```

或者通过回退PC的git版本到V2.38或早期版本也行，比如下：

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

7.6 固件的打包

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到`output/firmware`目录下：

固件生成：

```
./build.sh firmware
```

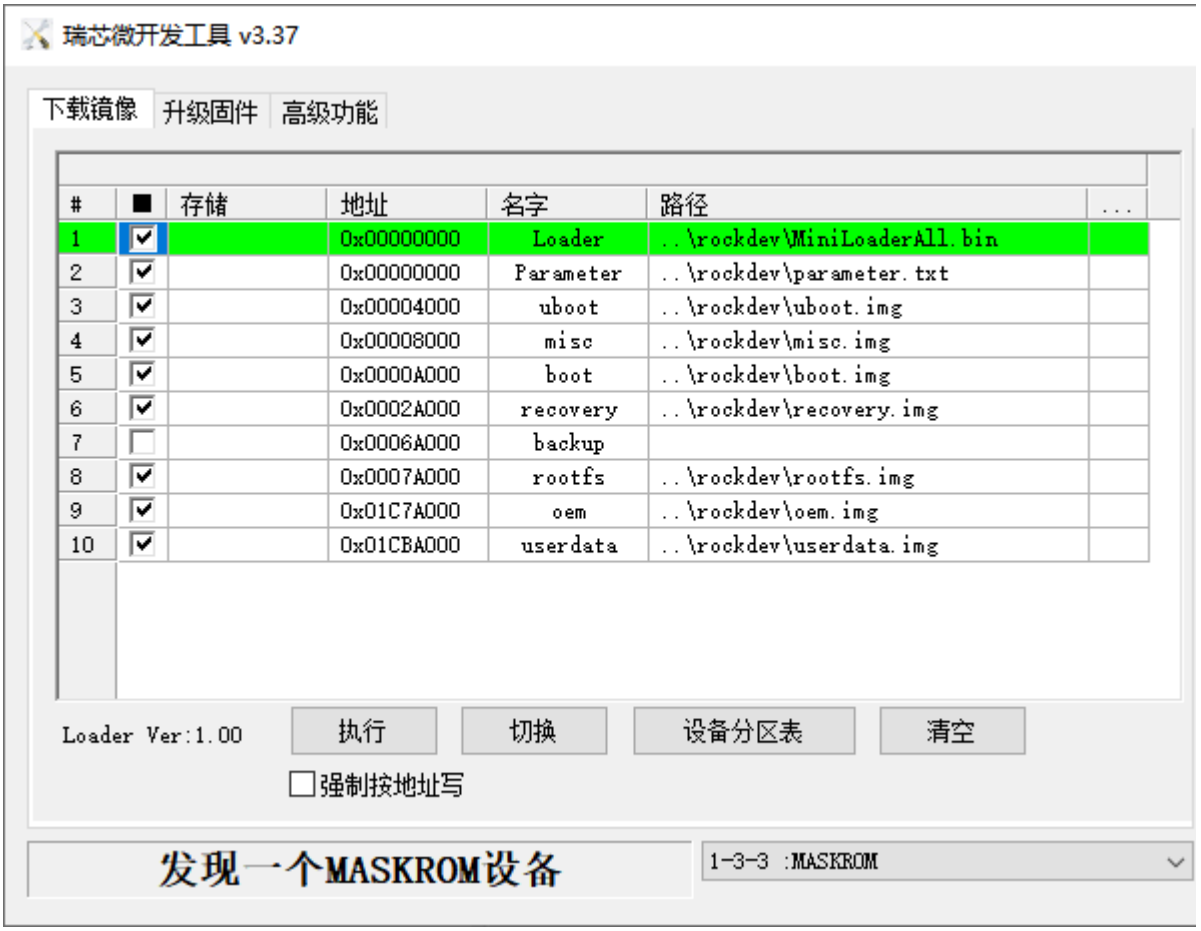
8. 刷机说明

8.1 Windows 刷机说明

SDK 提供 Windows 烧写工具（工具版本需要 V3.28 或以上），工具位于工程根目录：

```
tools/
  ├─── windows/RKDevTool
```

如下图，编译生成相应的固件后，设备烧写需要进入 MASKROM 或 BootROM 烧写模式，连接好 USB 下载线后，按住按键“MASKROM”不放并按下复位键“RST”后松手，就能进入MASKROM 模式，加载编译生成固件的相应路径后，点击“执行”进行烧写，也可以按 “recovery"按键不放并按下复位键 “RST” 后松手进入 loader 模式进行烧写，下面是 MASKROM 模式的分区偏移及烧写文件。
（注意：Windows PC 需要在管理员权限运行工具才可执行）



> 注：烧写前，需安装最新 USB 驱动，驱动详见：

```
<SDK>/tools/windows/DriverAssitant_v5.14.zip
```

8.2 Linux 刷机说明

Linux 下的烧写工具位于 tools/linux 目录下（Linux_Upgrade_Tool 工具版本需要 V2.36 或以上），请确认你的板子连接到 MASKROM/loader rockusb。比如编译生成的固件在 rockdev 目录下，升级命令如下：

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

或升级打包后的完整固件：

```
sudo ./upgrade_tool uf rockdev/update.img
```

或在根目录，机器在 MASKROM 状态运行如下升级：

```
./rkflash.sh
```

8.3 系统分区说明

默认分区说明（下面是 RK3572 EVB 分区参考）

Number	Start	End	Size	Name
1	8389kB	16.8MB	8389kB	uboot
2	16.8MB	21.0MB	4194kB	misc
3	21.0MB	88.1MB	67.1MB	boot
4	88.1MB	222MB	134MB	recovery
5	222MB	256MB	33.6MB	backup
6	256MB	15.3GB	15.0GB	rootfs
7	15.3GB	15.4GB	134MB	oem
8	15.4GB	62.5GB	47.1GB	userdata

- uboot 分区：供 uboot 编译出来的 uboot.img。
- misc 分区：供 misc.img，给 recovery 使用。
- boot 分区：供 kernel 编译出来的 boot.img。
- recovery 分区：供 recovery 编译出的 recovery.img。
- backup 分区：预留，暂时没有用。
- rootfs 分区：供 buildroot、debian 或 yocto 编出来的 rootfs.img。
- oem 分区：给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。
- userdata 分区：供 APP 临时生成文件或给最终用户使用，挂载在 /userdata 目录下。